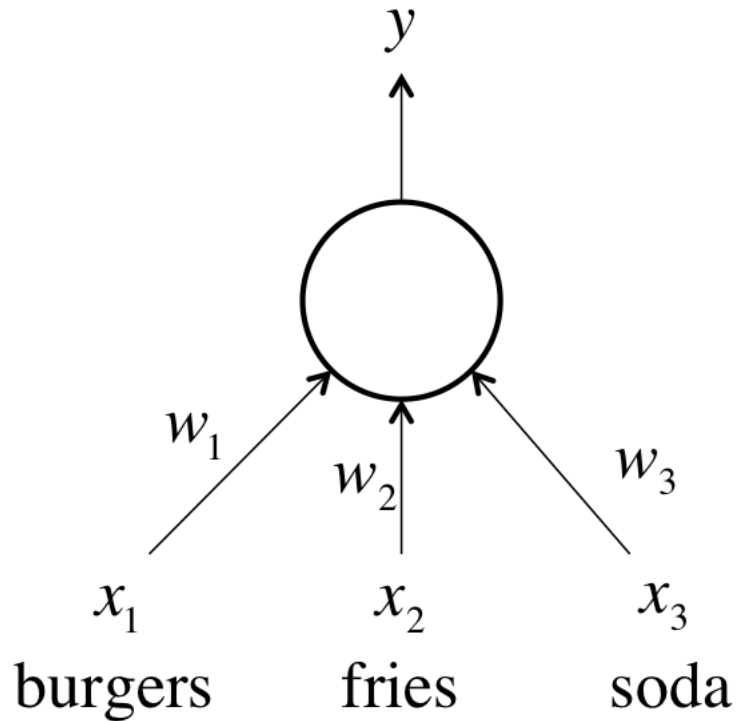


Training Feed-Forward Neural Networks

Unit 3.1

MDS 655

Fast Food Problem



- Every single day, we purchase a restaurant meal consisting of burgers, fries, and sodas
- We buy some number of servings for each item
- We want to be able to predict how much a meal is going to cost us, but the items don't have price tags.

$$y^{(i)} = w_1 x_1^{(i)} + w_2 x_2^{(i)} + w_3 x_3^{(i)}$$

Fast Food Problem (continued...)

How to solve fast food problem?

1. Preparing appropriate training set. (mixture of different servings like 1-1-1 for breakfast, lunch, dinner or 1-2-1, 2-1-2, 3-1-2 and so on...)
2. Adjustment of parameter vector θ (weights)
3. Minimization of error

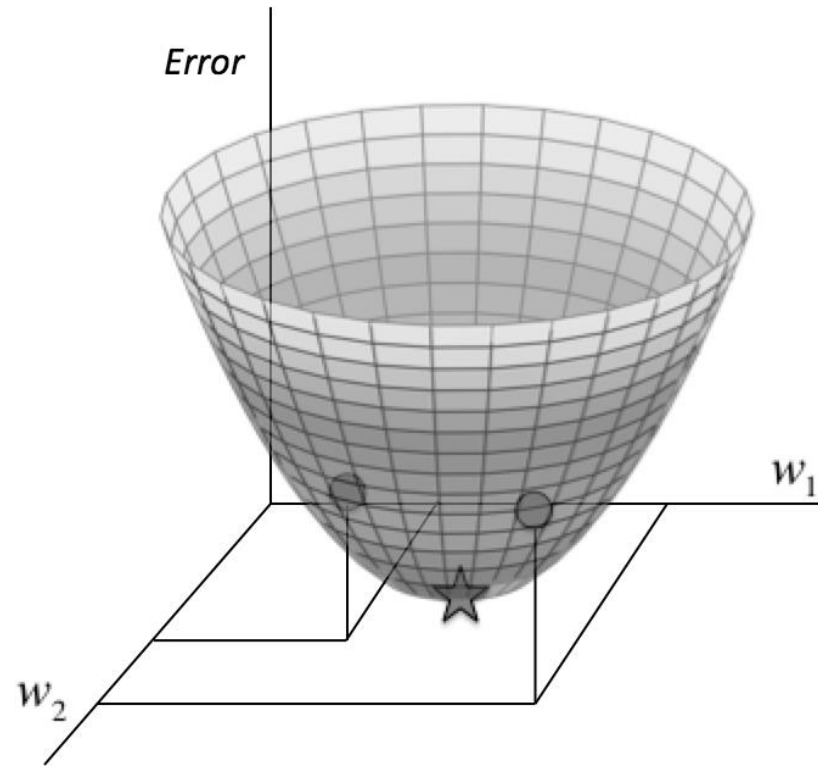
Fast Food Problem (continued...)

- we want to minimize the value of the error function E

$$E = \frac{1}{2} \sum_i (t^{(i)} - y^{(i)})^2$$

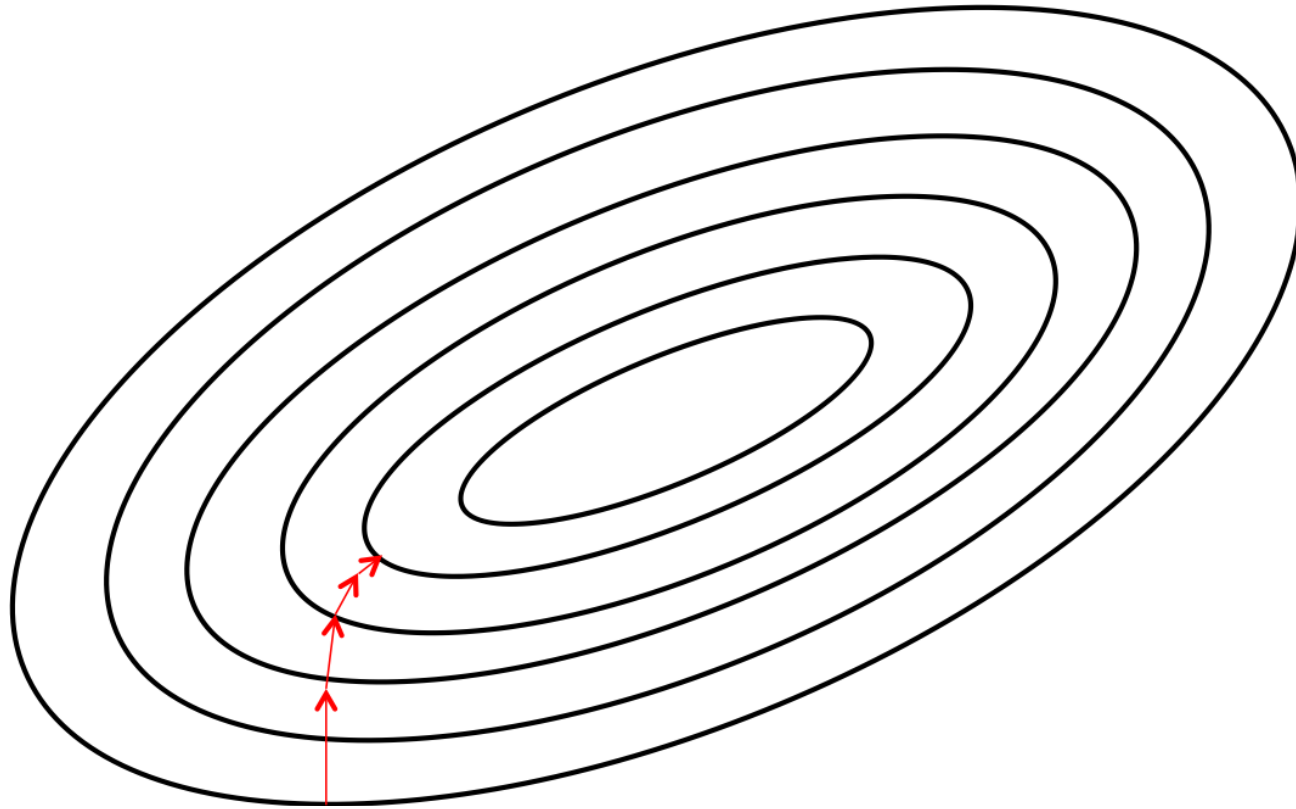
- The squared error is zero when our model makes a perfectly correct prediction on every training example.
- Moreover, the closer E is to 0, the better our model is.
- Parameter vector ϑ (the values for all the weights in our model) should be chosen such that E is as close to 0 as possible.

Gradient Descent



Gradient Descent (Continued...)

Visualizing the error surface as a set of contours



- Contours correspond to settings of w_1 and w_2 that evaluate to the same value of E .
- The closer the contours are to each other
 - the steeper the slope
 - direction of the steepest descent is always perpendicular to the contours.
 - This direction is expressed as a vector known as the gradient.

Gradient Descent (Continued...)

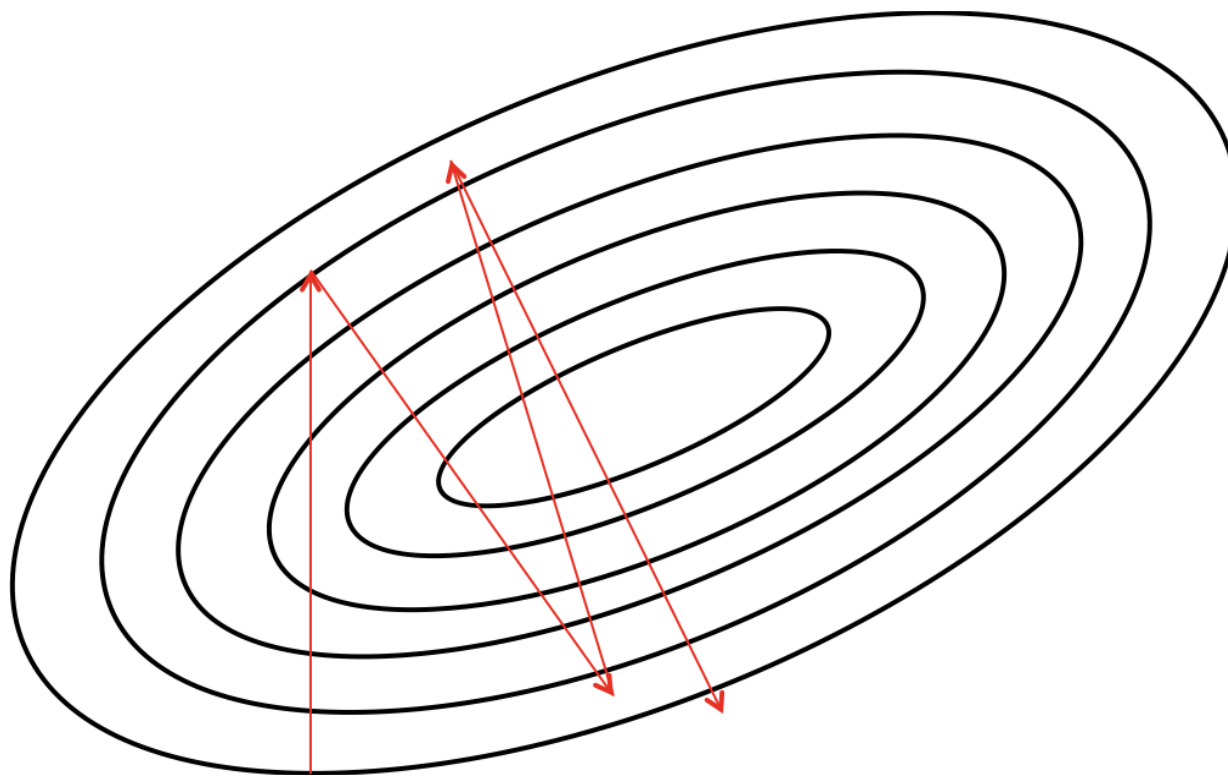
Gradient Descent Algorithm

- Randomly initialize the weights of our network so we find ourselves somewhere on the horizontal plane.
- By evaluating the gradient at our current position
 - we can find the direction of steepest descent
 - and we can take a step in that direction.

Gradient Descent (Continued...)

- Then we'll find ourselves at a new position that's closer to the minimum than we were before.
- We can reevaluate the direction of steepest descent
 - by taking the gradient at this new position
 - and taking a step in this new direction
- Following this strategy will eventually get us to the point of minimum error.

The Delta Rule and Learning Rates



The Delta Rule and Learning Rates (Continued...)

Hyperparameter: Learning Rate

- At each step of moving perpendicular to the contour, we need to determine
 - how far we want to walk before recalculating our new direction.
- Why this distance needs to depend on the steepness of the surface?

The Delta Rule and Learning Rates (Continued...)

- The closer we are to the minimum, the shorter we want to step forward and vice versa.
- We know we are close to the minimum
 - because the surface is a lot flatter,
- So we can use the steepness as an indicator of how close we are to the minimum.

The Delta Rule and Learning Rates (Continued...)

- However, if our error surface is rather mellow
 - Training can potentially take a large amount of time by shorter step
- As a result, we often multiply the gradient
 - by a factor , the learning rate ϵ

The Delta Rule and Learning Rates (Continued...)

Picking the learning rate is a hard problem

- If we pick a learning rate that's too small
 - we risk taking too long during the training process
- But if we pick a learning rate that's too big
 - we'll mostly likely start diverging away from the minimum

The Delta Rule and Learning Rates (Continued...)

- Therefore various optimization techniques
 - that utilize adaptive learning rates to automate the process of selecting learning rates is required

The Delta Rule and Learning Rates (Continued...)

Delta rule for training

- In order to calculate how to change each weight, we evaluate the gradient
- It is essentially the partial derivative of the error function with respect to each of the weights

The Delta Rule and Learning Rates (Continued...)

$$\begin{aligned}\Delta w_k &= -\epsilon \frac{\partial E}{\partial w_k} \\ &= -\epsilon \frac{\partial}{\partial w_k} \left(\frac{1}{2} \sum_i \left(t^{(i)} - y^{(i)} \right)^2 \right) \\ &= \sum_i \epsilon \left(t^{(i)} - y^{(i)} \right) \frac{\partial y_i}{\partial w_k} \\ &= \sum_i \epsilon x_k^{(i)} \left(t^{(i)} - y^{(i)} \right)\end{aligned}$$

Delta Rule Derivation

- **Objective:** Minimize the error function E for a single neuron.
- The error function is typically defined as the mean squared error (MSE):

$$E = \frac{1}{2} \sum_i (d_i - y_i)^2$$

where:

- d_i is the desired output for the i -th training example.
- y_i is the actual output for the i -th training example.

Delta Rule Derivation (continued...)

Output Calculation:

The actual output calculated using the activation function.

For simplicity, assume a linear activation function:

$$y_i = \sum_j w_j x_{ij}$$

where:

- w_j are the weights.
- x_{ij} are the input features for the i -th training example.

Delta Rule Derivation (continued...)

Error Gradient:

- To minimize the error E , we need to adjust the weights
- This is done using gradient descent
- The gradient of the error with respect to a weight w_j

$$\frac{\partial E}{\partial w_j} = \frac{\partial E}{\partial y_i} \cdot \frac{\partial y_i}{\partial w_j}$$

Delta Rule Derivation (continued...)

Partial Derivatives:

- The partial derivative of the error with respect to the output y_i

$$\frac{\partial E}{\partial y_i} = -(d_i - y_i)$$

- The partial derivative of the output y_i with respect to the weight w_j

$$\frac{\partial y_i}{\partial w_j} = x_{ij}$$

Delta Rule Derivation (continued...)

Combining the Derivatives:

$$\frac{\partial E}{\partial w_j} = -(d_i - y_i) \cdot x_{ij}$$

Delta Rule Derivation (continued...)

Weight Update Rule:

Using gradient descent, the weights are updated in the direction that reduces the error.

The learning rate ϵ/η controls the step size of the update:

$$\Delta w_j = -\eta \frac{\partial E}{\partial w_j} = \eta(d_i - y_i)x_{ij}$$

Delta Rule Derivation (continued...)

Final Delta Rule Equation:

- Therefore, the weight update rule, known as the Delta Rule, is:

$$\Delta w_j = \eta(d_i - y_i)x_{ij}$$

- The learning rate η is a critical parameter that determines the size of the weight updates
 - If η is too small, the training process will be slow
 - If η is too large, the algorithm may oscillate and fail to converge

Delta Rule Derivation (continued...)

- The Delta Rule is a straightforward method for updating weights in a neural network
- By iteratively applying the rule to each weight
 - the network's error can be minimized
 - leading to improved performance in prediction tasks
- This rule forms the basis for more complex learning algorithms in neural networks

The Back Propagation Algorithm

Look at Slide 1.1

Batch Gradient Descent

Batch Gradient Descent

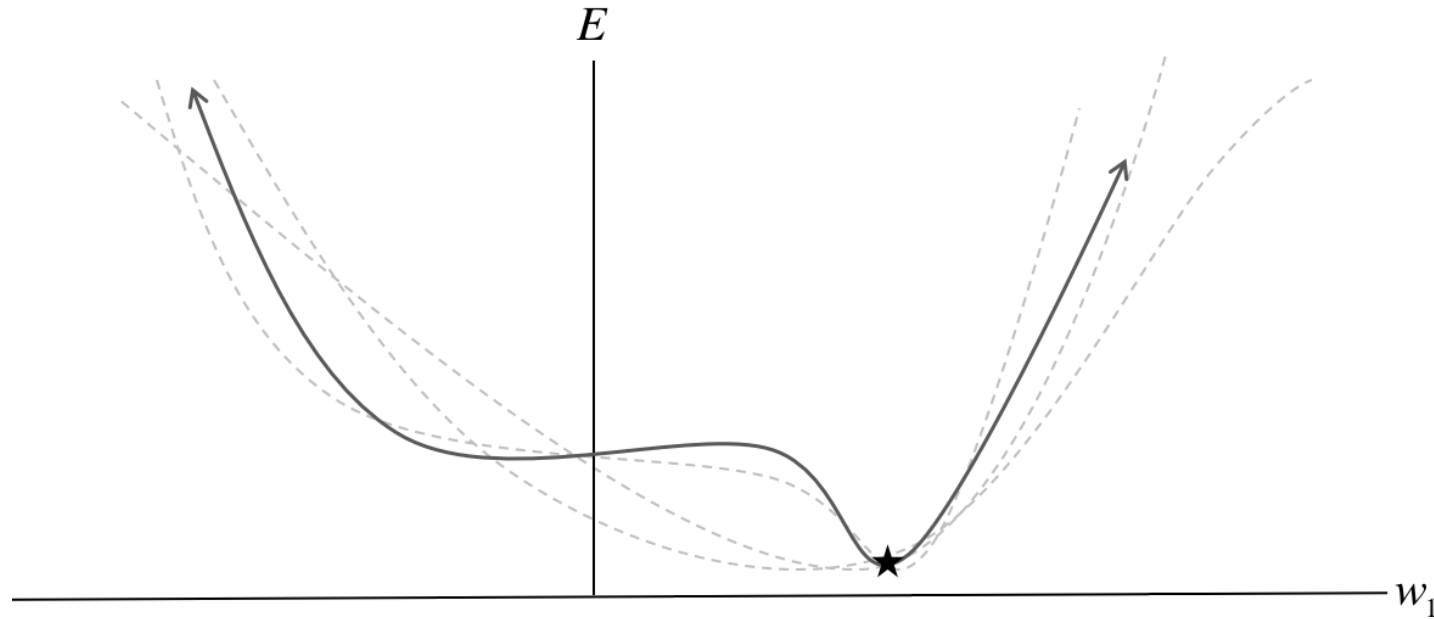
- The idea behind batch gradient descent is that
 - we use our entire dataset to compute the error surface
 - and then follow the gradient to take the path of steepest descent
- For a simple quadratic error surface, this works quite well
- But in most cases, our error surface may be a lot more complicated

Stochastic Gradient Descent

Stochastic Gradient Descent

- At each iteration
 - our error surface is estimated only with respect to a single example
 - where instead of a single static error surface, our error surface is dynamic
- As a result, descending on this stochastic surface significantly improves our ability to navigate flat regions

Stochastic Gradient Descent (Continued...)



The stochastic error surface fluctuates with respect to the batch error surface, enabling saddle point avoidance

Stochastic Gradient Descent (Continued...)

- The major pitfall of stochastic gradient descent
 - looking at the error incurred one example at a time may not be a good enough approximation of the error surface
- This, in turn, could potentially make gradient descent take a significant amount of time

Minibatch Gradient Descent

Minibatch Gradient Descent

- In mini-batch gradient descent, at every iteration
 - we compute the error surface with respect to some subset of the total dataset (instead of just a single example)
 - This subset is called a *minibatch*, and in addition to the learning rate, minibatch size is another hyperparameter

$$\Delta w_{ij} = - \sum_{k \in minibatch} \epsilon y_i^{(k)} y_j^{(k)} (1 - y_j^{(k)}) \frac{\partial E^{(k)}}{\partial y_j^{(k)}}$$

- Here, instead of summing over all the examples in the dataset
- we sum over the examples in the current minibatch